

Reduced Order Modeling for Parametric Navier–Stokes: A Comparison of POD-Galerkin, PODNN and PINN Approaches

Letizia Greco s336195
Course of Model Order Reduction and Machine Learning
Politecnico di Torino

June 12, 2026

1 Problem statement

We consider the stationary parametric Navier–Stokes problem on $\Omega = (0, 1)^2$: given the parameter $\boldsymbol{\mu} = (\mu_0, \mu_1) \in \mathcal{P} = [0.1, 10] \times [1, 3]$, find the velocity $\mathbf{u}(\boldsymbol{\mu})$ and pressure $p(\boldsymbol{\mu})$ such that

$$\begin{cases} -\mu_0 \nabla \cdot (\nabla \mathbf{u}) + (\nabla \mathbf{u}) \mathbf{u} + \nabla p = \mathbf{f}(\mathbf{x}; \mu_1) & \text{in } \Omega, \\ \nabla \cdot \mathbf{u} = 0 & \text{in } \Omega, \\ \mathbf{u} = \mathbf{0} & \text{on } \partial\Omega, \\ p(0, 0) = 0, \end{cases} \quad (1)$$

where μ_0 plays the role of kinematic viscosity and μ_1 modulates both the spatial frequency and the amplitude of the forcing term. The forcing $\mathbf{f} = (f_1, f_2)$ is defined as

$$\begin{aligned} f_1 &= -(\mu_1^3 \pi^2 \cos(\mu_1^2 \pi x) - \mu_1^2 \pi^2) \sin(\mu_1 \pi y) \cos(\mu_1 \pi y) + (\mu_1 \pi \cos(\mu_1 \pi x) \cos(\mu_1 \pi y)), \\ f_2 &= -(-\mu_1^3 \pi^2 \cos(\mu_1^2 \pi y) + \mu_1^2 \pi^2) \sin(\mu_1 \pi x) \cos(\mu_1 \pi x) + (-\mu_1 \pi \sin(\mu_1 \pi x) \sin(\mu_1 \pi y)). \end{aligned}$$

Two features make this problem non-trivial in the reduced-basis framework:

- The convective term $(\nabla \mathbf{u}) \mathbf{u}$ is **nonlinear**, requiring Newton iterations at each parameter query. Moreover, the resulting solution manifold has a **slow decay of the Kolmogorov n -width**, meaning that more reduced modes are needed to achieve a given accuracy than for the corresponding linear (Stokes) problem.
- The forcing $\mathbf{f}(\mathbf{x}; \mu_1)$ involves expressions such as $\cos(\mu_1^2 \pi x)$, so μ_1 is **not** affine in $\mathbf{f}$. Standard POD-Galerkin therefore cannot achieve a fully N_h -independent online cost.

Weak formulation. Multiplying (1) by suitable test functions and integrating by parts, we seek $(\mathbf{u}, p) \in V \times Q$ with $V = [H_0^1(\Omega)]^2$, $Q = L^2(\Omega)$ with $p(0, 0) = 0$, such that for all $(\mathbf{v}, q) \in V \times Q$:

$$\mu_0 \int_{\Omega} \nabla \mathbf{u} : \nabla \mathbf{v} \, d\Omega + \int_{\Omega} (\nabla \mathbf{u}) \mathbf{u} \cdot \mathbf{v} \, d\Omega - \int_{\Omega} p (\nabla \cdot \mathbf{v}) \, d\Omega = \int_{\Omega} \mathbf{f} \cdot \mathbf{v} \, d\Omega, \quad (2)$$

$$\int_{\Omega} (\nabla \cdot \mathbf{u}) q \, d\Omega = 0. \quad (3)$$

2 Full Order Model

Discretization. We discretize (2)–(3) using **Taylor–Hood** finite elements ($\mathbb{P}^2$ velocity, $\mathbb{P}^1$ pressure) on a triangular mesh of size $h = 0.001$. This choice satisfies the inf-sup condition at the discrete level. After assembly, the algebraic system reads

$$\underbrace{\begin{pmatrix} \mu_0 A & -B^\top \\ -B & 0 \end{pmatrix}}_{=: J_S(\mu_0)} \begin{pmatrix} \mathbf{u}_h \\ p_h \end{pmatrix} + \begin{pmatrix} \mathbf{c}(\mathbf{u}_h) \\ \mathbf{0} \end{pmatrix} = \begin{pmatrix} \mathbf{f}_h(\mu_1) \\ \mathbf{0} \end{pmatrix}, \quad (4)$$

where A is the velocity stiffness, B encodes the velocity-pressure coupling and $\mathbf{c}(\mathbf{u}_h)$ is the discrete convective operator (quadratic in $\mathbf{u}_h$).

Newton iteration. Equation (4) is solved at each μ via Newton’s method. Setting $\mathbf{u}^{k+1} = \mathbf{u}^k + \delta \mathbf{u}$ and linearizing the convective term via the product rule,

$$(\nabla \mathbf{u}^{k+1}) \mathbf{u}^{k+1} \approx (\nabla \mathbf{u}^k) \mathbf{u}^k + \underbrace{(\nabla \delta \mathbf{u}) \mathbf{u}^k}_{C_1(\mathbf{u}^k) \delta \mathbf{u}} + \underbrace{(\nabla \mathbf{u}^k) \delta \mathbf{u}}_{C_2(\mathbf{u}^k) \delta \mathbf{u}},$$

which yields

$$[J_S + J_C(\mathbf{u}^k)] \delta \mathbf{u} = \mathbf{f}_h - \mathbf{c}(\mathbf{u}^k) - J_S \mathbf{u}^k, \quad \mathbf{u}^{k+1} = \mathbf{u}^k + \delta \mathbf{u}, \quad (5)$$

where $J_C(\mathbf{u}^k) = C_1(\mathbf{u}^k) + C_2(\mathbf{u}^k)$ is the Newton Jacobian of the convective term. The method converges quadratically and typically requires 3–5 iterations to reach a tolerance of 10^{-8} in our experiments.

Operator structure. A key observation for ROM construction is that $J_S(\mu_0) = \mu_0 A + J_{BB}$ is *affine* in μ_0 , where J_{BB} collects the parameter-independent saddle-point blocks $-B$ and $-B^\top$ coupling velocity and pressure. Hence A and J_{BB} can be pre-assembled once and reused. By contrast, $\mathbf{f}_h(\mu_1)$ depends *non-affinely* on μ_1 and must be re-assembled for every new parameter, paying an $O(N_h)$ cost.

Consistency. The consistency of the FOM discretization is verified by a convergence study using a manufactured solution for the Navier–Stokes equations with $\nu = 0.1$ (Figure 1). Both velocity and pressure L^2 errors decrease with mesh refinement, confirming the correctness of the Taylor–Hood $\mathbb{P}^2$ – $\mathbb{P}^1$ implementation.

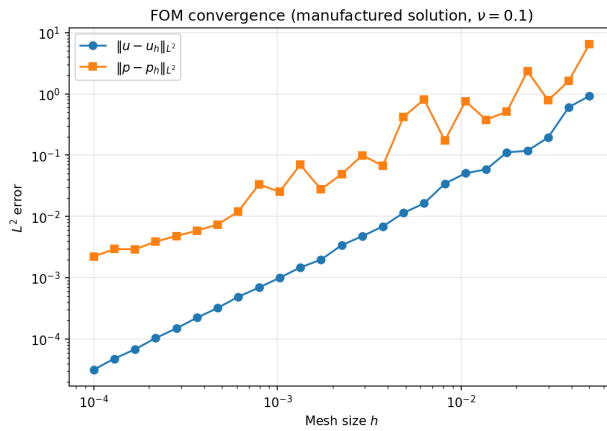


Figure 1: FOM convergence study using a manufactured exact solution for the Navier–Stokes equations with $\nu = 0.1$.

3 POD-Galerkin Reduced Order Model

3.1 Snapshot generation and supremizer enrichment

We sample $M = N_{\mu_0} \times N_{\mu_1} = 15 \times 30 = 450$ parameter values on a tensor grid with logarithmic spacing in μ_0 (which spans two orders of magnitude) and uniform spacing in μ_1 . For each $\boldsymbol{\mu}^{(m)}$, the FOM produces velocity and pressure snapshots $\mathbf{u}^{(m)}, p^{(m)}$.

We know from theory that a naive partitioned POD basis would fail to satisfy the inf-sup condition at the reduced level. To recover stability, we enrich the velocity basis with *inexact supremizer* snapshots, computed for each pressure snapshot):

$$X_u \mathbf{s}^{(m)} = B^\top p^{(m)}, \quad (6)$$

where X_u is the velocity inner-product matrix (H^1 seminorm). The factorization of X_u is computed once and reused for all M supremizer solves.

3.2 Partitioned POD

We perform three independent PODs with appropriate inner products: X_u for both velocity and supremizers, M_p (the pressure mass matrix) for pressure. For each snapshot family the correlation matrix is diagonalized and modes are retained according to a relative energy criterion with tolerance 10^{-7} . The eigenvalue decay (Figure 2) is rapid for all three blocks, confirming the reducibility of the problem.

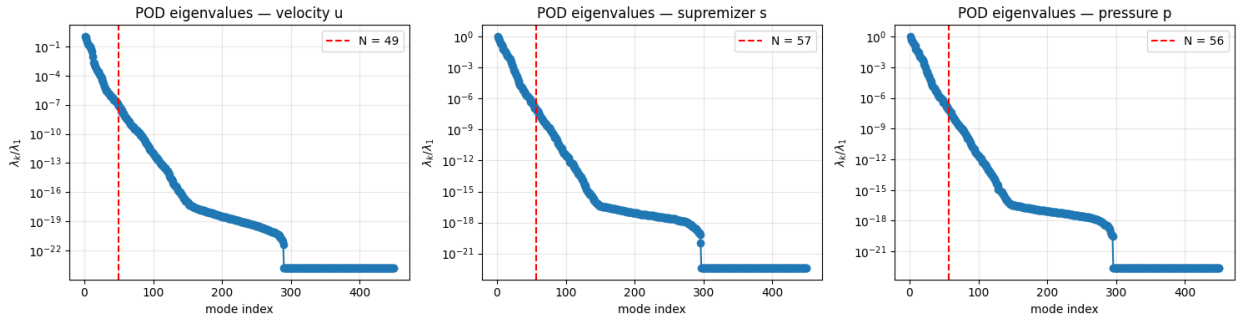


Figure 2: Normalized POD eigenvalues for velocity, supremizers, and pressure snapshots. Vertical dashed lines mark the number of modes retained by the relative energy criterion ($\varepsilon = 10^{-7}$).

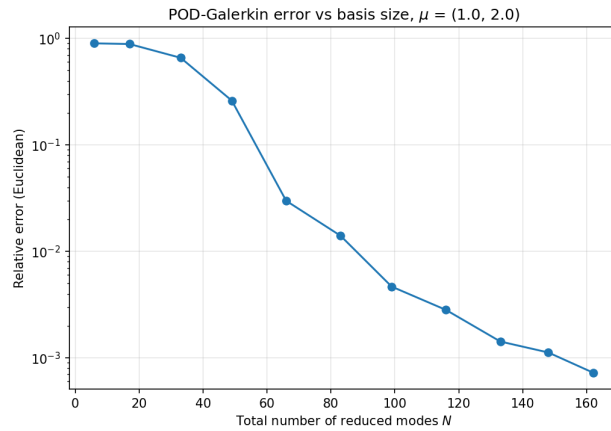


Figure 3: POD-Galerkin relative error $\frac{\|\mathbb{B}\boldsymbol{\alpha}(N) - \mathbf{u}_h^{\text{FOM}}\|_2}{\|\mathbf{u}_h^{\text{FOM}}\|_2}$ as a function of the total number of reduced modes N , for a representative test parameter $\boldsymbol{\mu} = (1.0, 2.0)$. The error decreases monotonically, confirming that the POD basis progressively captures the solution structure.

3.3 Reduced Newton solver

Given $\mathbb{B}$, the reduced system is obtained by Galerkin projection. The μ -independent operators are pre-projected offline:

$$A_N = \mathbb{B}^\top A \mathbb{B}, \quad J_{BB,N} = \mathbb{B}^\top J_{BB} \mathbb{B}.$$

For each new μ , online we:

1. Assemble $\mathbf{f}_h(\mu_1)$ in FE space (cost $O(N_h)$) and project: $\mathbf{f}_N = \mathbb{B}^\top \mathbf{f}_h$.
2. Solve the reduced nonlinear system via Newton on $\mathbb{R}^N$. At each iteration, $c(\mathbf{u}^k)$ and J_C are reassembled in FE space at the reconstructed velocity $\mathbb{B}\boldsymbol{\alpha}^k$, then projected to the reduced space. This “reassemble-and-project” strategy is simple to implement but its online cost still scales with N_h , limiting the achievable speedup, as discussed in Section 6.

4 PODNN: a neural-network surrogate

PODNN bypasses the reduced solve at the online stage by directly learning the map

$$\pi^{\text{NN}} : \mu \mapsto \boldsymbol{\alpha}(\mu) \in \mathbb{R}^N,$$

i.e. the vector of reduced coefficients associated with the FOM solution. Once trained, evaluating the network at a new μ requires only a forward pass, with cost **independent of** N_h .

Target coefficients. For each training snapshot, the target reduced coefficient is the Galerkin projection of the FOM solution onto the reduced basis:

$$\boldsymbol{\alpha}^{(m)} = (\mathbb{B}^\top X \mathbb{B})^{-1} \mathbb{B}^\top X u_h^{(m)}, \quad X = \text{diag}(X_u, M_p).$$

Since $\mathbb{B}$ is X -orthonormal by construction, $\mathbb{B}^\top X \mathbb{B} = I$ and the formula simplifies to $\boldsymbol{\alpha}^{(m)} = \mathbb{B}^\top X u_h^{(m)}$.

Architecture and training. We use a feed-forward network with 5 hidden layers of 150 neurons each and tanh activations. Training uses Adam with an initial learning rate of 10^{-3} , `ReduceLROnPlateau` scheduler (factor 0.5, patience 3000), and gradient clipping (max norm 1.0) over 100 000 epochs. The MSE loss on the training set decreases below 10^{-5} , indicating sufficient capacity.

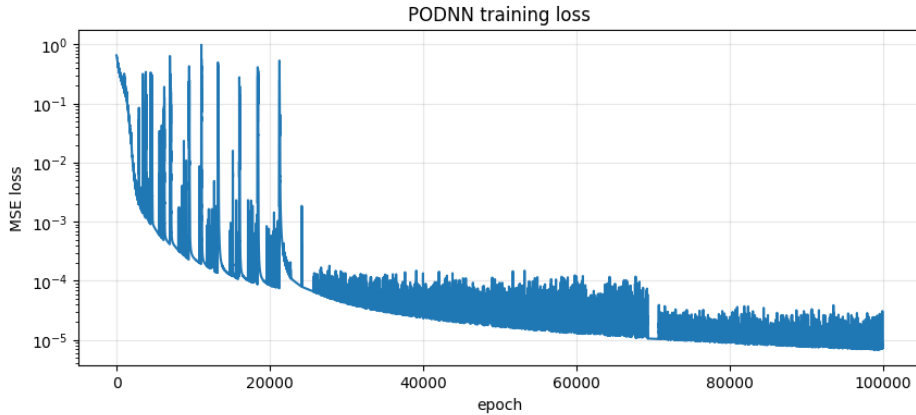


Figure 4: PODNN training loss (MSE on POD coefficients) over 100 000 epochs. The loss reaches $\sim 10^{-5}$ and plateaus after approximately 40 000 epochs, with periodic drops corresponding to learning rate reductions by the `ReduceLROnPlateau` scheduler.

5 PINN: a Physics-Informed Neural Network

As an alternative to the projection-based approaches, we consider a Physics-Informed Neural Network (PINN) that solves the parametric Navier–Stokes problem *without any FOM data*. The network takes (x, y, μ_0, μ_1) as input and returns (u_x, u_y, p) as output, learning the solution map directly from the governing equations and boundary conditions.

Architecture. We use the same architecture adopted for PODNN (5 hidden layers of 150 neurons, tanh activations) to allow a fair comparison. Since $\mu_0 \in [0.1, 10]$ spans two orders of magnitude, we normalize it internally via a log-scale mapping to $[0, 1]$; μ_1 is mapped linearly. The PDE residual is always evaluated with the physical parameter values.

Loss function and sampling. The training loss is $\mathcal{L} = \mathcal{L}_{\text{bc}} + \lambda \mathcal{L}_{\text{pde}} + \mathcal{L}_{p_0}$, where $\mathcal{L}_{\text{pde}}$ is the mean squared residual of the momentum and incompressibility equations at $N_p = 3000$ interior collocation points, $\mathcal{L}_{\text{bc}}$ enforces $\mathbf{u} = \mathbf{0}$ on $\partial\Omega$ at $N_b = 400$ boundary points, and $\mathcal{L}_{p_0}$ imposes $p(0, 0; \boldsymbol{\mu}) = 0$ at $N_{p_0} = 20$ random parameter pairs. All points are resampled uniformly at each epoch (Monte Carlo), making the optimization stochastic. Computing the PDE residual requires 10 automatic differentiation calls per epoch (6 first-order and 4 second-order derivatives), which is the main computational bottleneck.

Training and hyperparameter selection. We use Adam with initial learning rate 10^{-3} , with the ReduceLROnPlateau scheduler (factor 0.5, patience 3000), and gradient clipping (max norm 1.0) for 100 000 epochs. We tested $\lambda \in \{0.1, 0.3, 0.5, 1.0, 10.0\}$.

Error metrics for the PINN. Unlike the ROM-based methods, the PINN produces a continuous function on $\Omega \times \mathcal{P}$ with no associated FEM discretization, so the X -norm is not directly applicable. To enable a fair comparison, both the PINN and the FOM are evaluated on the FEM mesh. For each of the 20 test parameters $(\mu_0^{(k)}, \mu_1^{(k)}) \in \mathcal{P}$, the FOM solution is extracted on the mesh vertices, yielding nodal values $(u^{\text{FOM}}, p^{\text{FOM}})$. The trained PINN is then queried at the same nodes with parameter inputs $(\mu_0^{(k)}, \mu_1^{(k)})$, producing $(\tilde{u}, \tilde{p})$. We report the relative ℓ^2 errors

$$\text{err}_{\text{vel}}^{(k)} = \frac{\|\tilde{u} - u^{\text{FOM}}\|_2}{\|u^{\text{FOM}}\|_2}, \quad \text{err}_p^{(k)} = \frac{\|\tilde{p} - p^{\text{FOM}}\|_2}{\|p^{\text{FOM}}\|_2}, \quad (7)$$

averaged over the 20 test parameters.

6 Numerical results

6.1 Setup

The FOM uses Taylor–Hood $\mathbb{P}^2$ – $\mathbb{P}^1$ elements on a mesh of size $h = 0.001$, yielding $N_h = 6,761$ degrees of freedom.

The training set for POD–Galerkin and PODNN consists of $15 \times 30 = 450$ snapshots on a grid with logarithmic spacing in μ_0 and uniform spacing in μ_1 . The offline snapshot generation takes 552s (≈ 9 min). The partitioned POD with relative energy tolerance $\varepsilon = 10^{-7}$ retains $N_u = 49$ velocity modes, $N_s = 57$ supremizer modes, and $N_p = 56$ pressure modes, for a total reduced dimension of $N = 162$.

All methods are evaluated on the same test set of 20 random parameters drawn uniformly from $\mathcal{P}$ with seed 42, filtered to ensure a minimum normalized distance of 0.02 from any training point. We report relative errors in the Euclidean norm and, for POD–Galerkin and PODNN, also in the energy norm $\|\cdot\|_X = (\|\cdot\|_{H^1, \text{vel}}^2 + \|\cdot\|_{L^2, \text{pres}}^2)^{1/2}$. For the PINN, we report velocity and pressure errors separately.

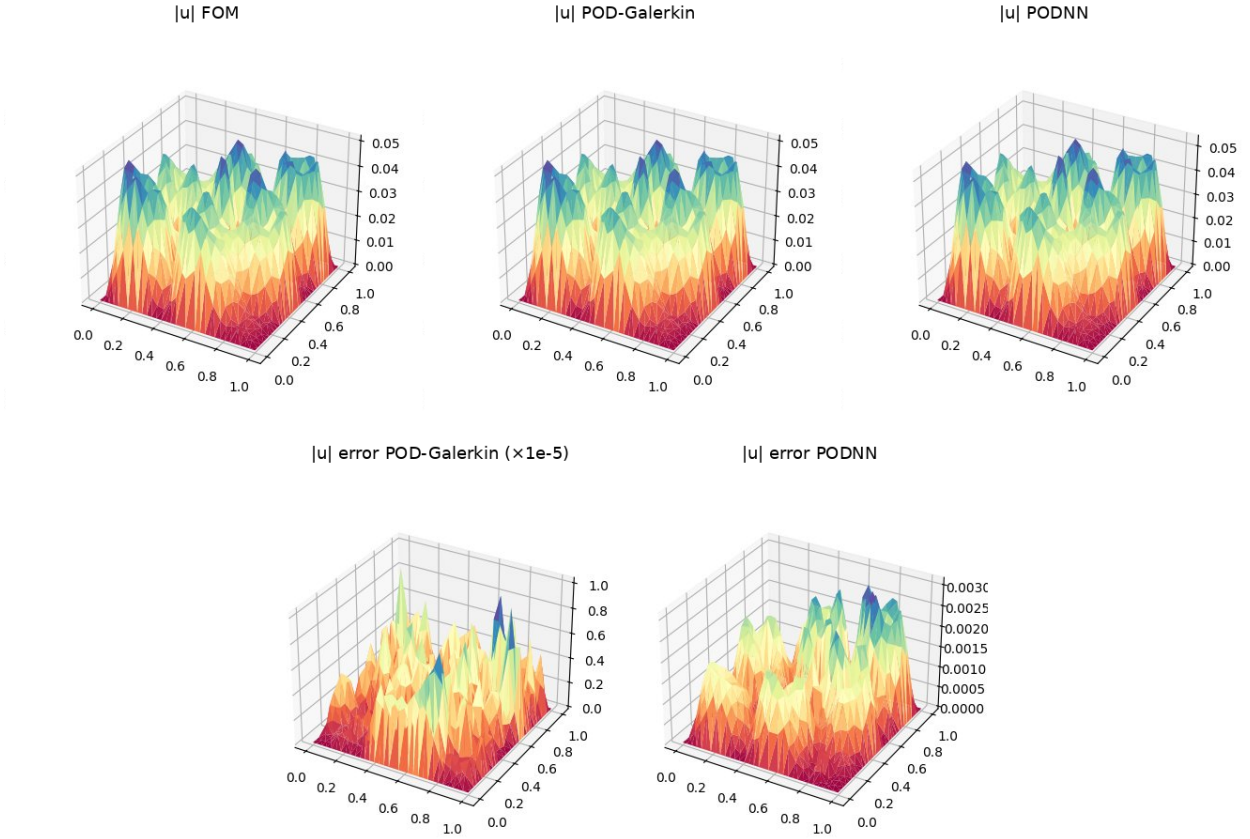


Figure 5: Top row: velocity magnitude $|\mathbf{u}|$ for $\mu = (4.44, 2.45)$ computed by FOM, POD-Galerkin, and PODNN. Both reduced methods are visually indistinguishable from the FOM. Bottom row: pointwise error $||\mathbf{u}|_{\text{FOM}} - |\mathbf{u}|_{\text{approx}}|$. The POD-Galerkin error is of order 10^{-5} , while the PODNN error reaches $\sim 3 \times 10^{-3}$, consistent with the relative errors reported in Table 1.

6.2 Accuracy comparison

Table 1: Mean relative errors and speedups over 20 test parameters.

Method	$\overline{\text{err}}_2$	$\overline{\text{err}}_X$	$\overline{\text{err}}_{\text{vel}}$	$\overline{\text{err}}_p$	speedup
POD-Galerkin	3.87×10^{-4}	3.63×10^{-4}	—	—	$1.4 \times$
PODNN	2.61×10^{-2}	7.25×10^{-2}	—	—	$17.5 \times$
PINN ($\lambda = 0.1$)	—	—	7.33×10^{-1}	1.32	$467 \times$

Three orders of magnitude separate the POD-Galerkin error ($\sim 4 \times 10^{-4}$) from the PINN velocity error ($\sim 7 \times 10^{-1}$), with PODNN sitting in between ($\sim 3 \times 10^{-2}$). The gap between POD-Galerkin and PODNN is entirely due to the neural network approximation of the coefficient map $\mu \mapsto \alpha$: both methods share the same POD basis and are therefore subject to the same projection error floor.

The X -norm errors for PODNN (7.25×10^{-2}) are roughly three times larger than the Euclidean ones (2.61×10^{-2}). This is expected: the H^1 seminorm in the velocity component penalizes gradient errors, which are harder to approximate in a linear subspace for advection-dominated problems with slow Kolmogorov n -width decay.

The PINN velocity errors exceed 1 for some test parameters, particularly at high μ_0 combined with high μ_1 (e.g., $\mu = (9.4, 2.8)$ with $\text{err}_{\text{vel}} = 1.42$). Pressure errors are consistently above 1, reflecting the difficulty of learning the pressure field over the full parameter range.

6.3 Effect of λ in the PINN

Table 2: PINN mean errors over 20 test parameters for different λ values.

λ	$\overline{\text{err}}_{\text{vel}}$	$\overline{\text{err}}_p$
0.1	7.33×10^{-1}	1.32
0.3	8.36×10^{-1}	1.03
0.5	1.16	1.37
1.0	1.33	1.20
10.0	3.06	0.86

The velocity error increases monotonically with λ : smaller values emphasize boundary enforcement, producing a better-constrained velocity field. The pressure error, however, follows the opposite trend, with $\lambda = 10.0$ achieving the lowest value (0.86). This asymmetry comes from the loss structure: velocity is enforced as a full Dirichlet condition on $\partial\Omega$, while pressure has only a single point condition and is learned almost exclusively through the PDE residual (where ∇p appears). Strengthening the PDE penalty thus benefits the pressure but weakens boundary enforcement for the velocity — hence the opposite trends.

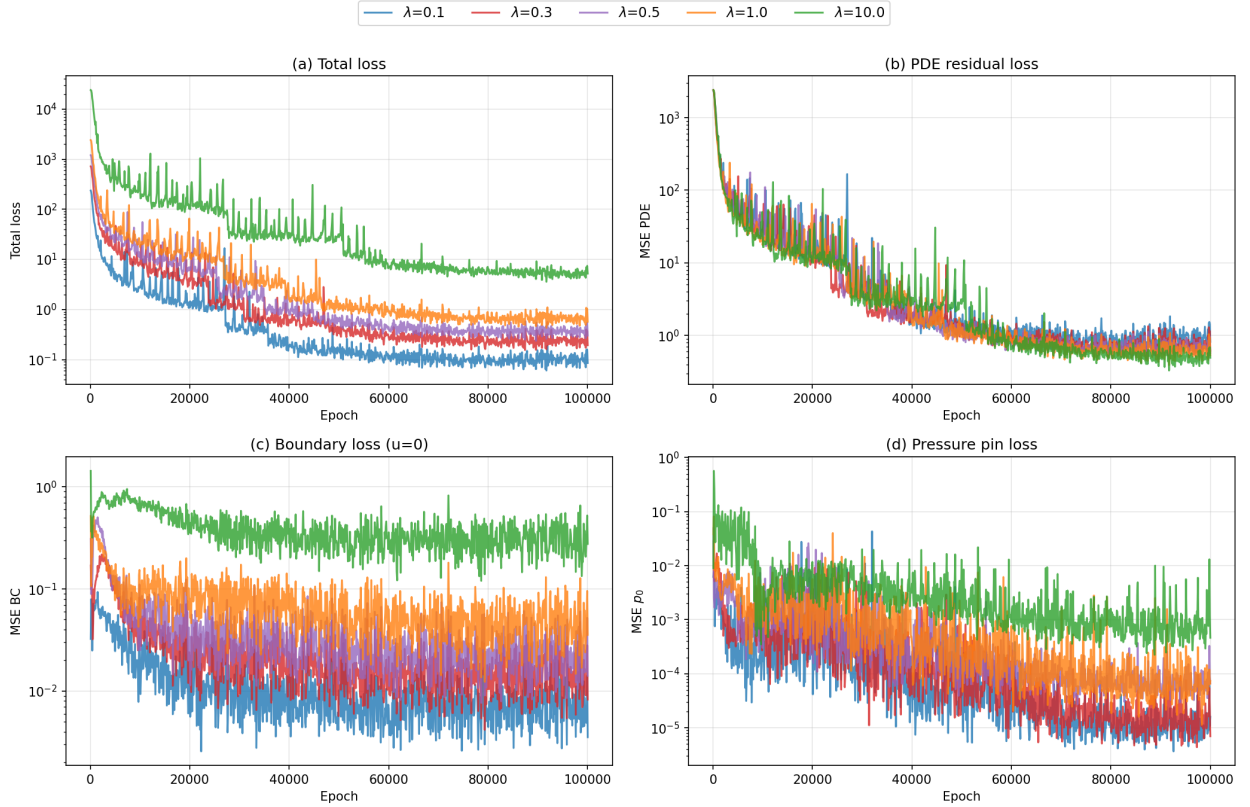


Figure 6: PINN training losses for different λ values: (a) total loss, (b) PDE residual, (c) boundary loss, (d) pressure pin loss. Smaller λ values lead to lower boundary losses but higher PDE residuals, confirming the trade-off discussed in the text.

6.4 Computational costs

The POD-Galerkin ROM achieves a modest speedup of $1.4\times$ because the non-affine forcing and the convective term must be reassembled in the full-order space at each Newton iteration, resulting in an online cost that still scales with N_h .

Table 3: Computational cost breakdown.

	Offline	Online (per μ)
FOM	—	~ 1.4 s
POD-Galerkin	552 s (snapshots)	~ 1 s
PODNN	552 s + 861 s (training)	~ 0.08 s
PINN ($\lambda = 0.1$)	~ 550 min (training)	~ 0.003 s

PODNN eliminates this bottleneck entirely: a forward pass through the network costs ~ 0.08 s (with speedups between $\sim 9.12\times$ and $\sim 91.7\times$, depending on the parameters), independent of N_h . Its offline cost is higher but this would be amortized in the many-query regime.

The PINN requires no offline snapshot generation, but its training cost significantly exceeds the combined offline cost of PODNN. Once trained, evaluation is fast (< 0.01 s), but the accuracy gap with the other methods limits its practical utility for this problem.

7 Conclusions

We have implemented and compared three approaches for a parametric Navier–Stokes problem with non-affine forcing: POD-Galerkin ROM, PODNN, and a parametric PINN.

POD-Galerkin achieves the best accuracy ($\sim 4 \times 10^{-4}$) by directly projecting the FOM equations onto a reduced basis, but the non-affine forcing and the nonlinear convective term prevent a fully N_h -independent online cost, limiting the speedup to $\sim 1.4\times$.

PODNN replaces the reduced solver with a neural network that maps parameters to POD coefficients. The accuracy is lower ($\sim 3 \times 10^{-2}$) due to the additional approximation error, but the online evaluation is entirely independent of N_h , yielding a significant speedup ($\sim 17.5\times$). For applications requiring many parameter queries, this trade-off is favorable.

The PINN solves the problem without any FOM data, using only the PDE residual and boundary conditions as training signal. While this makes it the most flexible approach, the accuracy is significantly lower ($\sim 7 \times 10^{-1}$ for velocity), and the training cost significantly exceeds the offline phase of the other methods. The difficulty of balancing the loss components (governed by λ) and the wide parametric range (μ_0 spanning two orders of magnitude) could be the main limiting factors.

Future improvements could include EIM to recover an affine decomposition of the forcing in POD-Galerkin, adaptive loss balancing strategies for the PINN, exploration of alternative network architectures and the use of quadrature rules for the loss integrals of the PINN.